

Ros Day04 hw2 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.14

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제는 ROS 2 와 C++을 사용해 Lifecycle Node 기반 Publisher·Subscriber 를 구현하고, QoS 와 자동·수동 상태 전환을 적용하는 것이었다. AI 는 코드를 작성하는 과정과 코드를 실행시키는데에 사용했다.

2. AI 사용 부분

먼저 숫자를 주고받는 간단한 예제로 과제를 구현하기로 정했다. AI 의 도움으로 하나의 패키지에 Publisher 와 Subscriber 를 만들고, 자동 상태 전환용 `auto.launch.py` 와 수동 상태 전환용 `manual.launch.py` 를 각각 구성했다.

그렇다면 거리 센서 설정까지 넣을 필요 없겠네. **숫자 카운터를 주고받는 가장 단순한 예제로 가자.**

- Publisher: 1초마다 1, 2, 3... 을 `/count` 토픽에 발행
- Subscriber: 받은 숫자를 터미널에 출력
- 메시지: `std_msgs/Int32`
- QoS: 양쪽 모두 `Keep Last`, `Depth 10`, `Reliable`, `Volatile`

Lifecycle은 **Active일 때만 발행·수신 내용을 처리하고, Inactive에서는 멈추는 모습**을 보여주는 데 집중하면 돼. 보고서에는 “카운터에 Lifecycle이 꼭 필요한 것은 아니지만, 상태 전환에 따라 통신을 제어하는 방법을 검증하는 학습용 예제”라고 솔직하게 설명하면 좋겠어. ROS 2 공식 예제도 간단한 `talker/listener`로 이 동작을 보여줘. [Lifecycle 공식 예제](#)

좋아. C++ 패키지 하나에 Lifecycle 노드 두 개를 넣는 방향으로 정하자.

노드	역할	Active일 때
<code>counter_publisher</code>	<code>/count</code> 에 <code>std_msgs/Int32</code> 발행	1초마다 숫자 1씩 증가해 발행
<code>counter_subscriber</code>	<code>/count</code> 구독	받은 숫자를 로그로 출력

과제 1에서 “적합한 패키지”는 `rclcpp_lifecycle` 을 중심으로 설명하면 돼. C++에서 `LifecycleNode` 와 상태별 Callback을 제공하고, `LifecyclePublisher` 는 비활성 상태에서 발행되지 않도록 관리할 수 있어. Subscriber의 메시지 처리 중단은 상태에 맞춰 **직접 구현해야 한다**는 점도 보고서에 적으면 좋아. [rclcpp_lifecycle 문서](#), [LifecyclePublisher API](#)

QoS는 두 노드 모두 `Keep Last / Depth 10 / Reliable / Volatile` 로 시작하자. 숫자가 작고 1초에 한 번만 오므로 전달 신뢰성을 우선할 수 있고, 늦게 켜 Subscriber에게 이전 숫자를 다시 보여줄 필요는 없어. 이는 ROS 2 의 기본 QoS 조합이기도 해. [ROS 2 QoS 문서](#)

보고서에서는 “카운터 자체에 Lifecycle이 필수”라고 주장하기보다, **상태 전환에 따라 발행·처리가 시작되고 멈추는 것을 확인하기 위한 예제**라고 설명하자.

또한 수동 launch 파일을 실행시킬때 AI 를 사용하였다.

이번에는 **수동 Launch가 정상 실행됐어**. 노드 두 개가 시작됐지만 숫자가 나오지 않는 건 아직 **Unconfigured** 상태라서 맞는 동작이야.

화면 위쪽의 **Package not found** 는 처음에 ROS 환경을 불러오기 전에 실행해서 나온 오류야. 그다음 **source** 를 실행한 뒤에는 해결됐어. 보고서에는 나중에 오류 문구가 없는 화면을 캡처하면 돼.

다음 한 단계: 지금 터미널은 그대로 두고, 새 터미널을 열어 두 노드의 상태를 확인해줘.

</> Bash

```
source /opt/ros/jazzy/setup.bash
ros2 lifecycle get /counter_publisher
ros2 lifecycle get /counter_subscriber
```

둘 다 **unconfigured [1]** 로 나오면 그 결과를 보내줘. 그다음에 직접 **configure** 로 바꿔보자.



맞아. 두 노드 모두 **unconfigured [1]** 이야. 프로세스는 실행됐지만 통신 준비는 아직 하지 않은 상태라는 뜻이야. 이 화면도 수동 Launch의 시작 상태를 보여주는 증거로 쓸 수 있어.

이제 새 터미널에서 **configure**만 해보자.

</> Bash

```
ros2 lifecycle set /counter_subscriber configure
ros2 lifecycle set /counter_publisher configure
ros2 lifecycle get /counter_subscriber
ros2 lifecycle get /counter_publisher
```

예상 결과는 둘 다 **inactive [2]** 야. 이때도 ↓ 는 아직 발행되지 않아야 해. 결과를 보내주면 **configure** 에서 코드가 실제로 무엇을 준비했는지 짚어볼게.

3. 깨달은 점

처음에는 Launch 파일을 실행하면 바로 숫자를 주고받을 것이라고 생각했다. 하지만 수동 Launch에서는 두 노드가 unconfigured 상태로 시작했고, configure 후에도 inactive 상태라 숫자가 출력되지 않았다. activate 까지 진행한 뒤에야 발행과 수신이 시작되는 것을 보며, 노드를 실행하는 것과 실제 동작을 시작하는 것이 다르다는 점을 이해했다.

또한 Publisher를 비활성화하면 숫자 발행 자체가 멈췄다. 반면 Subscriber를 비활성화하면 Publisher는 계속 숫자를 발행하지만, Subscriber의 수신 출력만 멈췄다. 다시 활성화했을 때는 그 시점부터 받은 숫자가 출력됐다. 이를 통해 각 노드의 Lifecycle 상태를 따로 제어할 수 있다는 것을 알게 되었다.